MOTOR_WS_GUIDE

motor_ws — EtherCAT 모터 제어 시스템 가이드

> ROS2 기반 EtherCAT 듀얼 갠트리 로봇 6축 모터 제어 시스템
> 드라이버: LS Mecapion iX7NH Series (CiA 402)

목차

1. 프로젝트 개요
2. 디렉토리 구조
3. 패키지 설명
4. 시스템 아키텍처
5. 빌드 방법
6. 테스트 모드 설정
7. 실행 방법
8. 원점 복귀 (Homing)
9. ROS2 인터페이스 레퍼런스
10. 설정 파라미터
11. 사용 예시
12. 의존성 및 요구사항
13. 안전 기능

---

1. 프로젝트 개요

`motor_ws` 는 **단일 EtherCAT 버스**를 통해 **두 갠트리(총 6축)** 의 서보 드라이버를 단일 ROS2 노드로 실시간 제어하는 워크스페이스입니다.

하드웨어 구성

갠트리	축	모터 수	이동 범위	리드	리밋 스위치
갠트리 0	X	1개	0 ~ 1750 mm	11.9993 mm/rev	NOT (역방향)
갠트리 0	Z	2개 (동기)	-1750 ~ 0 mm	5.9996 mm/rev	POT (정방향)
갠트리 1	X	1개	0 ~ 1750 mm	11.9993 mm/rev	NOT (역방향)
갠트리 1	Z	2개 (동기)	-1750 ~ 0 mm	5.9996 mm/rev	POT (정방향)

- **엔코더 분해능:** 8,388,608 pulse/rev (2^{23}, 전기적 기어비 1:1)
- **제어 주기:** 10 ms (기본값) / 1 ms 권장 (실시간 커널 필요)
- **통신 방식:** EtherCAT — CiA 402 프로파일 (단일 체인, 6 슬레이브)

제어 모드

모드	CiA 402 코드	설명
CSV (Cyclic Synchronous Velocity)	Mode 9	일반 이동 — 사다리꼴 속도 프로파일
Homing	Mode 6	원점 복귀 — 리밋 스위치 + 인덱스 펄스

원점 복귀 방법 (축별 공통)

축	Method	방향	기준 스위치	최종 원점
X축	**1**	역방향 (−)	NOT (Negative Over Travel)	Index (Z) 펄스
Z축	**2**	정방향 (+)	POT (Positive Over Travel)	Index (Z) 펄스

---

2. 디렉토리 구조

```
motor_ws/
├── src/
│   ├── motor_control/                      # 메인 제어 패키지 (Python)
│   │   ├── motor_control/
│   │   │   ├── __init__.py
│   │   │   ├── motor_driver_node.py        # ROS2 메인 노드 (듀얼 갠트리)
│   │   │   ├── ethercat_interface.py       # EtherCAT 버스 통신 핵심
│   │   │   ├── homing_action_server.py     # 원점 복귀 액션 서버
│   │   │   └── move_action_server.py       # 이동 액션 서버
│   │   ├── launch/
│   │   │   ├── motor_control.launch.py     # 메인 런치 파일
│   │   │   └── homing_only.launch.py       # 원점 복귀 전용 런치 파일
│   │   ├── config/
│   │   │   └── motor_params.yaml           # 설정 파라미터 (6슬레이브, 갠트리별)
│   │   ├── package.xml
│   │   ├── setup.py
│   │   └── setup.cfg
│   │
│   └── motor_control_interfaces/           # 인터페이스 정의 패키지 (CMake)
│       ├── msg/
│       │   └── AxisState.msg
│       ├── srv/
│       │   ├── SetVelocity.srv
│       │   └── GetDriveStatus.srv          # gantry_index 필드 포함
│       ├── action/
│       │   ├── Homing.action               # gantry_index 필드 포함
│       │   └── MoveAxis.action             # gantry_index + 갠트리별 목표 포함
│       ├── package.xml
│       └── CMakeLists.txt
│
├── DUAL_GANTRY_GUIDE.md                    # 듀얼 갠트리 사용 가이드
```

```
├── README.md
├── VELOCITY_GUIDE.md
├── TEST_X_AXIS.md
└── TEST_Z_AXIS.md
```

---

3. 패키지 설명

3-1. `motor_control_interfaces`

커스텀 ROS2 메시지·서비스·액션 타입 정의 패키지 (C++ CMake 빌드).

타입	이름	주요 변경
msg	`AxisState`	갠트리당 1개 발행 (`/motor/gantry0/` , `/motor/gantry1/`)
srv	`SetVelocity`	변경 없음 (두 갠트리 공통 적용)
srv	`GetDriveStatus`	요청에 `gantry_index` 추가
action	`Homing`	goal에 `gantry_index` 추가 (0/1/2)
action	`MoveAxis`	goal에 `gantry_index` + 갠트리별 목표 필드 추가

3-2. `motor_control`

Python으로 작성된 메인 제어 패키지.

`ethercat_interface.py` — EtherCAT 통신 핵심

- `pysoem` 라이브러리로 EtherCAT 슬레이브 6대와 통신
- 별도 **서브프로세스**에서 제어 루프 실행 (1 kHz 목표)
- CiA 402 상태 머신 구현 (INIT → PRE-OP → SAFE-OP → OP)
- 호밍 완료 감지: `bit12_cleared` 추적으로 잔류 플래그 오탐 방지
- `z_pairs` : 여러 Z1-Z2 쌍을 독립적으로 Cross-Coupling 동기화

핵심 상수:

```
EFFECTIVE_PPR  = 8_388_608        # pulse/rev (전기적 기어비 1:1)
X_MM_PER_REV   = 11.9993131404    # mm/rev (X축 볼스크류)
Z_MM_PER_REV   = 5.99965657019    # mm/rev (Z축 볼스크류)
```

EtherCATInterface 생성 시 주요 파라미터:

```
EtherCATInterface(
    adapter_name=ifname,
    num_slaves=6,
    z_pairs=[(z1_g0, z2_g0), (z1_g1, z2_g1)],  # 갠트리별 Z 쌍 목록
    ...
)
```

`motor_driver_node.py` — ROS2 메인 노드

- 갠트리별 슬레이브 인덱스를 `gantry0_*`, `gantry1_*` 파라미터로 관리
- `/motor/gantry0/axis_state`, `/motor/gantry1/axis_state` 토픽 발행
- `_make_axis_state(gantry_idx)` 메서드로 갠트리별 상태 생성
- `get_gantry_indices()` → `{0: (xi, z1i, z2i), 1: (xi, z1i, z2i)}`
- `get_soft_limits()` → 갠트리별 소프트 리밋 딕셔너리

`homing_action_server.py` — 원점 복귀 서버

- `goal.gantry_index` : 0=갠트리0, 1=갠트리1, 2=동시 호밍
- `gantry_indices` 딕셔너리로 각 갠트리 슬레이브 인덱스 관리
- 비활성 축(idx=-1)은 자동으로 완료 상태 처리
- `bit12_cleared` + `homing_b4_cycles` 로 안정적인 rising edge 확보

`move_action_server.py` — 이동 액션 서버

- `goal.gantry_index` 로 이동 대상 갠트리 결정
  - `0` → `gantry0_x/z_target_mm` 사용
  - `1` → `gantry1_x/z_target_mm` 사용
  - `2` → `gantry2_x/z_target_mm` 를 두 갠트리에 공통 적용
- 갠트리별 독립 소프트 리밋 검증
- 갠트리별 Z1-Z2 동기화 오차 실시간 모니터링

---

4. 시스템 아키텍처

```
┌──────────────────────────────────────────────────────┐
│                 메인 프로세스 (ROS2)                  │
│                                                      │
│  MotorDriverNode                                     │
│  ├── pub: /motor/gantry0/axis_state (10Hz)           │
│  ├── pub: /motor/gantry1/axis_state (10Hz)           │
│  ├── HomingActionServer  (/motor/homing)             │
│  │     gantry_index: 0 / 1 / 2(동시)                 │
│  ├── MoveAxisActionServer (/motor/move)              │
│  │     gantry_index: 0 / 1 / 2(동시)                 │
│  ├── /motor/set_velocity  (service, 공통)            │
│  └── /motor/get_status    (service, gantry_index)    │
│                                                      │
│         ↕ 커맨드 큐 (mp.Queue)                       │
│         ↕ 공유 메모리 (mp.Array, 슬레이브×8슬롯)      │
└──────────────────────────────────────────────────────┘
                          │
┌──────────────────────────────────────────────────────┐
│             EtherCAT 프로세스 (제어 루프)             │
│                                                      │
│  _ethercat_process_loop()                            │
```

```
│   ├── Slave 0: 갠트리0 X축  (idx=0)                │
│   ├── Slave 1: 갠트리0 Z1축 (idx=1)                │
│   ├── Slave 2: 갠트리0 Z2축 (idx=2)                │
│   ├── Slave 3: 갠트리1 X축  (idx=3)                │
│   ├── Slave 4: 갠트리1 Z1축 (idx=4)                │
│   └── Slave 5: 갠트리1 Z2축 (idx=5)                │
│                                                    │
│   Z Pair 0: (Slave 1, Slave 2) — 갠트리0 동기화    │
│   Z Pair 1: (Slave 4, Slave 5) — 갠트리1 동기화    │
│                                                    │
│   ※ idx=-1 슬레이브는 제어 루프에서 건너뜀          │
└────────────────────────────────────────────────────┘
                          │ EtherCAT 케이블 (단일 체인)
┌────────────────────────────────────────────────────┐
│          서보 드라이버 (iX7NH ×6)                   │
│   [0:G0-X] → [1:G0-Z1] → [2:G0-Z2]                 │
│           → [3:G1-X]  → [4:G1-Z1] → [5:G1-Z2]      │
└────────────────────────────────────────────────────┘
```

슬레이브별 공유 상태 슬롯 (슬레이브당 8슬롯)

슬롯	내용
0	Status Word (CiA 402)
1	이동 중 플래그 (0/1)
2	현재 위치 (pulse)
3	원점 오프셋 (pulse)
4	동기화 오차 플래그 (0/1)
5	현재 속도 (pulse/sec)
6	원점 복귀 완료 플래그 (0/1)
7	원점 복귀 단계 (0=idle / 1=진행중 / 2=완료 / -1=오류)

---

5. 빌드 방법

```
cd ~/motor_ws

# 인터페이스 먼저 빌드 (의존성 순서)
colcon build --packages-select motor_control_interfaces
colcon build --packages-select motor_control

# 또는 전체 빌드
colcon build --symlink-install

# 매 터미널마다 환경 소싱 필요
source install/setup.bash
```

6. 테스트 모드 설정

`motor_params.yaml` 에서 미사용 슬레이브를 `-1` 로 설정하고 `num_slaves` 를 실제 연결 대수로 조정합니다.

단계별 권장 순서

```
1단계: 갠트리0 X축 단독
2단계: 갠트리0 전체 (X+Z1+Z2)
3단계: 갠트리1 전체 (갠트리0 없이)
4단계: 듀얼 갠트리 전체
```

1단계 — 갠트리0 X축 단독

```yaml
num_slaves: 1
gantry0_x_slave_idx:  0
gantry0_z1_slave_idx: -1
gantry0_z2_slave_idx: -1
gantry1_x_slave_idx:  -1
gantry1_z1_slave_idx: -1
gantry1_z2_slave_idx: -1
```

2단계 — 갠트리0 전체

```yaml
num_slaves: 3
gantry0_x_slave_idx:  0
gantry0_z1_slave_idx: 1
gantry0_z2_slave_idx: 2
gantry1_x_slave_idx:  -1
gantry1_z1_slave_idx: -1
gantry1_z2_slave_idx: -1
```

3단계 — 갠트리1 전체 (갠트리0 없이)

```yaml
num_slaves: 3
gantry0_x_slave_idx:  -1
gantry0_z1_slave_idx: -1
gantry0_z2_slave_idx: -1
gantry1_x_slave_idx:  0
gantry1_z1_slave_idx: 1
gantry1_z2_slave_idx: 2
```

4단계 — 듀얼 갠트리 전체

```yaml
num_slaves: 6
gantry0_x_slave_idx:  0
gantry0_z1_slave_idx: 1
gantry0_z2_slave_idx: 2
```

```
gantry1_x_slave_idx:  3
gantry1_z1_slave_idx: 4
gantry1_z2_slave_idx: 5
```

---

7. 실행 방법

```
source ~/motor_ws/install/setup.bash

# 기본 실행
ros2 launch motor_control motor_control.launch.py

# 네트워크 인터페이스 직접 지정
ros2 launch motor_control motor_control.launch.py ifname:=enp3s0
```

실행 흐름

```
1. 하드웨어 전원 ON + EtherCAT 케이블 연결 확인
2. motor_params.yaml에서 테스트 모드 설정
3. colcon build --symlink-install && source install/setup.bash
4. sudo -s (root 필요)
5. ros2 launch motor_control motor_control.launch.py
6. /motor/homing 액션으로 원점 복귀 (gantry_index: 2)
7. /motor/move 액션으로 이동 명령
```

---

8. 원점 복귀 (Homing)

X축 — Method 1 (NOT + Index)

```
역방향(-) 이동 → NOT 리밋 스위치 감지 → 방향 전환 → Index 펄스 감지 → 원점 확정
```

Z축 — Method 2 (POT + Index)

```
정방향(+) 이동 → POT 리밋 스위치 감지 → 방향 전환 → Index 펄스 감지 → 원점 확정
```

호밍 안정화 로직 (bit12_cleared)

드라이브 이전 세션에서 bit12(Homing Attained)가 잔류할 수 있습니다.
다음 두 조건을 **모두** 만족할 때만 호밍 완료로 인정합니다:

1. `bit12_cleared = True` — 호밍 시작 후 bit12가 한 번 이상 0이었음
2. `homing_b4_cycles >= 3` — 3사이클 동안 bit4=0 유지

---

9. ROS2 인터페이스 레퍼런스

9-1. 토픽

`/motor/gantry0/axis_state`, `/motor/gantry1/axis_state`

타입: `motor_control_interfaces/msg/AxisState`

필드	타입	설명
`header`	std_msgs/Header	타임스탬프 / `frame_id: "gantry0"` 또는 `"gantry1"`
`x_position_mm`	float64	X축 현재 위치 [mm]
`z_position_mm`	float64	Z축 평균 위치 [mm] (Z1·Z2 평균)
`z1_position_mm`	float64	Z1 모터 개별 위치 [mm]
`z2_position_mm`	float64	Z2 모터 개별 위치 [mm]
`x_velocity_mm_s`	float64	X축 현재 속도 [mm/s]
`z_velocity_mm_s`	float64	Z축 현재 속도 [mm/s]
`z_sync_error_mm`	float64	Z1-Z2 동기화 오차 [mm]
`x_homed`	bool	X축 원점 복귀 완료
`z_homed`	bool	Z축 원점 복귀 완료
`drive_state`	string	CiA 402 상태 문자열
`sync_fault`	bool	동기화 오차 비상 정지 여부

9-2. 서비스

`/motor/get_status`

```
# 요청
int32 gantry_index   # 0=갠트리0, 1=갠트리1
---
# 응답
bool    ethercat_connected
string  cia402_state
bool    x_homed
bool    z_homed
float64 x_position_mm
float64 z_position_mm
string  error_code
bool    sync_fault
```

`/motor/set_velocity`

두 갠트리에 공통 적용됩니다.

```
# 요청
float64 velocity_mm_s   # 목표 속도 [mm/s]
string  axis            # "x" | "z" | "all"
---
bool   success
string message
```

9-3. 액션

`/motor/homing`

```
# Goal
int32 gantry_index   # 0=갠트리0, 1=갠트리1, 2=동시 호밍
bool  force_rehome
---
# Result
bool    success
string  message
float64 gantry0_z_home_position_mm
float64 gantry1_z_home_position_mm
---
# Feedback
string  phase                    # "in_progress" | "idle"
float64 gantry0_current_z_mm
float64 gantry1_current_z_mm
```

`/motor/move`

```
# Goal
int32   gantry_index

float64 gantry0_x_target_mm     # gantry_index=0 시 사용 (-1.0e9 = 현 위치 유지)
float64 gantry0_z_target_mm
float64 gantry1_x_target_mm     # gantry_index=1 시 사용
float64 gantry1_z_target_mm
float64 gantry2_x_target_mm     # gantry_index=2 시 두 갠트리 공통 적용
float64 gantry2_z_target_mm

float64 velocity_mm_s            # 0 = default_velocity_mm_s 사용
bool    force_move
---
# Result
bool    success
float64 gantry0_x_final_mm
float64 gantry0_z_final_mm
float64 gantry1_x_final_mm
float64 gantry1_z_final_mm
string  message
---
```

```
# Feedback
float64 gantry0_x_current_mm
float64 gantry0_z_current_mm
float64 gantry1_x_current_mm
float64 gantry1_z_current_mm
float64 gantry0_x_remaining_mm
float64 gantry0_z_remaining_mm
float64 gantry1_x_remaining_mm
float64 gantry1_z_remaining_mm
```

---

10. 설정 파라미터

파일: `src/motor_control/config/motor_params.yaml`

네트워크 / 하드웨어

파라미터	기본값	설명
`ifname`	`"enP8p1s0"`	EtherCAT 인터페이스 이름
`num_slaves`	`6`	총 슬레이브 수
`gantry0_x_slave_idx`	`0`	갠트리0 X축 슬레이브 (-1=비활성)
`gantry0_z1_slave_idx`	`1`	갠트리0 Z1축 슬레이브
`gantry0_z2_slave_idx`	`2`	갠트리0 Z2축 슬레이브
`gantry1_x_slave_idx`	`3`	갠트리1 X축 슬레이브
`gantry1_z1_slave_idx`	`4`	갠트리1 Z1축 슬레이브
`gantry1_z2_slave_idx`	`5`	갠트리1 Z2축 슬레이브

타이밍

파라미터	기본값	설명
`pdo_cycle_ms`	`10`	PDO 제어 주기 [ms] (1 ms 권장, RT 커널 필요)
`state_publish_hz`	`10.0`	AxisState 발행 주기 [Hz]

속도 / 가속도

파라미터	기본값	설명
`default_velocity_rpm`	`30.0`	기본 속도 [RPM]
`default_velocity_mm_s`	`75.0`	기본 속도 [mm/s] (goal=0 시 사용)
`default_accel_rpm_s`	`1000.0`	기본 가감속 [RPM/s]

호밍

파라미터	기본값	설명
`x_homing_method`	`1`	X축 호밍 메서드
`z_homing_method`	`2`	Z축 호밍 메서드
`homing_fast_rpm`	`30.0`	탐색 속도 [RPM]
`homing_slow_rpm`	`1.0`	Index 탐색 속도 [RPM]
`homing_timeout_s`	`1200.0`	호밍 타임아웃 [초] (두 갠트리 합산 기준)

Z축 동기화 (Cross-Coupling, 갠트리별 독립 적용)

파라미터	기본값	설명
`enable_coupling`	`true`	Z1-Z2 Cross-Coupling 활성화
`coupling_gain`	`0.05`	보정 게인 [1/s]
`max_sync_error_mm`	`10.0`	긴급정지 임계값 [mm]
`ma_window`	`5`	이동평균 윈도우 (사이클)

소프트웨어 리밋

파라미터	기본값	설명
`gantry0_x_soft_limit_min_mm`	`0.0`	갠트리0 X 하한 [mm]
`gantry0_x_soft_limit_max_mm`	`1750.0`	갠트리0 X 상한 [mm]
`gantry0_z_soft_limit_min_mm`	`-1750.0`	갠트리0 Z 하한 [mm]
`gantry0_z_soft_limit_max_mm`	`0.1`	갠트리0 Z 상한 [mm]
`gantry1_x_soft_limit_min_mm`	`0.0`	갠트리1 X 하한 [mm]
`gantry1_x_soft_limit_max_mm`	`1750.0`	갠트리1 X 상한 [mm]
`gantry1_z_soft_limit_min_mm`	`-1750.0`	갠트리1 Z 하한 [mm]
`gantry1_z_soft_limit_max_mm`	`0.1`	갠트리1 Z 상한 [mm]

---

11. 사용 예시

Python 클라이언트 — 두 갠트리 동시 제어

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from motor_control_interfaces.action import Homing, MoveAxis
from motor_control_interfaces.srv import GetDriveStatus

rclpy.init()
```

```python
node = Node('dual_gantry_client')

# ── 상태 조회 ──────────────────────────────────────────
status_cli = node.create_client(GetDriveStatus, '/motor/get_status')
status_cli.wait_for_service()

for gantry_idx in [0, 1]:
    req = GetDriveStatus.Request()
    req.gantry_index = gantry_idx
    res = status_cli.call(req)
    print(f"[갠트리{gantry_idx}] 연결={res.ethercat_connected}, "
          f"X={res.x_position_mm:.1f}mm, Z={res.z_position_mm:.1f}mm")

# ── 두 갠트리 동시 호밍 (gantry_index=2) ──────────────
homing_cli = ActionClient(node, Homing, '/motor/homing')
homing_cli.wait_for_server()

future = homing_cli.send_goal_async(
    Homing.Goal(gantry_index=2, force_rehome=False),
    feedback_callback=lambda fb: print(
        f"호밍 phase={fb.feedback.phase} "
        f"G0_Z={fb.feedback.gantry0_current_z_mm:.1f}mm "
        f"G1_Z={fb.feedback.gantry1_current_z_mm:.1f}mm"
    ),
)
rclpy.spin_until_future_complete(node, future)
result = future.result().get_result_async()
rclpy.spin_until_future_complete(node, result)
print(f"호밍 완료: {result.result().result.success}")

# ── 두 갠트리 동시·동일 이동 (gantry_index=2) ──────────
move_cli = ActionClient(node, MoveAxis, '/motor/move')
move_cli.wait_for_server()

goal = MoveAxis.Goal(
    gantry_index=2,
    gantry0_x_target_mm=-1.0e9, gantry0_z_target_mm=-1.0e9,
    gantry1_x_target_mm=-1.0e9, gantry1_z_target_mm=-1.0e9,
    gantry2_x_target_mm=500.0,
    gantry2_z_target_mm=-200.0,
    velocity_mm_s=50.0,
    force_move=False,
)
future = move_cli.send_goal_async(
    goal,
    feedback_callback=lambda fb: print(
        f"G0 X={fb.feedback.gantry0_x_current_mm:.1f} "
        f"G1 X={fb.feedback.gantry1_x_current_mm:.1f}"
    ),
)
rclpy.spin_until_future_complete(node, future)
result = future.result().get_result_async()
rclpy.spin_until_future_complete(node, result)
```

```
r = result.result().result
print(f"이동 완료: G0 X={r.gantry0_x_final_mm:.1f} / G1 X=
{r.gantry1_x_final_mm:.1f}")

node.destroy_node()
rclpy.shutdown()
```

---

12. 의존성 및 요구사항

```
sudo apt install ros-humble-rclpy ros-humble-std-msgs ros-humble-action-msgs
sudo pip3 install pysoem
```

항목	요구사항
OS	Linux Ubuntu 22.04 권장
ROS2	Humble 이상
Python	3.8 이상
커널	RT Kernel 권장 (1 ms 주기 시)
권한	CAP_NET_RAW — 실행 시 `sudo -s`
하드웨어	EtherCAT NIC × 1 + iX7NH 드라이버 × 1~6

---

13. 안전 기능

소프트 리밋 (갠트리별 독립)

```
갠트리0·1 공통:  X: 0.0 ~ 1750.0 mm / Z: -1750.0 ~ 0.1 mm
```

`force_move: true` 지정 시 소프트 리밋 건너뜀 (테스트·긴급 복구).

Z축 동기화 오차 감지 (갠트리별 독립)

각 갠트리의 Z1-Z2 이동 평균 차이가 `max_sync_error_mm` 초과 시
해당 갠트리 전체 비상 정지. 다른 갠트리는 계속 동작합니다.

원점 복귀 선행 조건

`/motor/move` 는 대상 갠트리의 모든 활성 축이 호밍 완료 후에만 허용됩니다 (`force_move: false` 기준).

CiA 402 드라이브 폴트 감지

매 PDO 사이클마다 Status Word 감시. Fault 비트 검출 시 해당 슬레이브 즉시 정지.

빠른 참조

```bash
# 환경 소싱
source ~/motor_ws/install/setup.bash

# 시스템 시작
sudo -s
ros2 launch motor_control motor_control.launch.py

# 상태 모니터링
ros2 topic echo /motor/gantry0/axis_state
ros2 topic echo /motor/gantry1/axis_state

ros2 service call /motor/get_status motor_control_interfaces/srv/GetDriveStatus \
  "{gantry_index: 0}"

# 두 갠트리 동시 호밍
ros2 action send_goal /motor/homing motor_control_interfaces/action/Homing \
  "{gantry_index: 2, force_rehome: false}"

# 두 갠트리 동시·동일 이동 (X=500mm, Z=-200mm)
ros2 action send_goal /motor/move motor_control_interfaces/action/MoveAxis "{
  gantry_index: 2,
  gantry0_x_target_mm: -1.0e9, gantry0_z_target_mm: -1.0e9,
  gantry1_x_target_mm: -1.0e9, gantry1_z_target_mm: -1.0e9,
  gantry2_x_target_mm: 500.0,  gantry2_z_target_mm: -200.0,
  velocity_mm_s: 50.0, force_move: false}"

# 속도 변경
ros2 service call /motor/set_velocity motor_control_interfaces/srv/SetVelocity \
  "{velocity_mm_s: 50.0, axis: 'all'}"
```